A Dual Active-Set Quadratic Programming Solver in NumPy and SciPy

Thomas Schmelzer
Jebel Quant Research
https://github.com/Jebel-Quant/quadprog

August 2026

Abstract

The Goldfarb–Idnani dual active-set method (Goldfarb and Idnani, 1983) remains one of the most effective algorithms for small and medium dense strictly convex quadratic programs, and the `quadprog` lineage descending from Powell's ZQPCVX (Powell, 1983) and Turlach's Fortran translation (Turlach et al., 2019) is still the reference implementation in both R and Python. That lineage is compiled code: it presumes a C toolchain and a build step. We describe a reimplementation in NumPy and SciPy alone, and use it as a controlled setting in which to ask which of the reference's implementation decisions are essential to the algorithm and which are artefacts of the language it was written in.

Three findings are reported. First, the reference's chain of Givens rotations for adding a constraint can be replaced by a single Householder reflection without altering the sequence of iterates, because the quantities the solver consumes are invariant to the choice of orthogonal reduction; we prove this and confirm it empirically on 3027 problems, where both implementations agree on every iteration count. Second, the reference's packed storage for the triangular factor, easily mistaken for a memory optimisation, is load-bearing for performance: it is what keeps the active submatrix contiguous and therefore admissible to a BLAS packed solve, worth a factor of 10.2 on that operation. Third, detecting that a constraint column holds a single nonzero — which is what a bound constraint is — reduces three per-iteration products from $O(n^2)$ or $O(nm)$ to indexing.

The result is faster than the compiled reference for $n \gtrsim 135$, by 11 times at $n = 1600$, while remaining slower for small problems by a factor set by interpreter dispatch rather than by arithmetic. Attacking that floor at its source — the *number* of iterations rather than their cost — by guessing the active set and certifying it against the KKT conditions moves the crossover to $n \approx 65$ and the margin to 68 times at $n = 1600$. We also report two negative results in full: leaving the orthogonal factor implicit in the manner of LAPACK's `geqrf`/`ormqr` is 2.4 to 2.6 times *slower*, and a Numba (Lam et al., 2015) port wins by an order of magnitude at $n = 10$ but loses by 2.5 times at $n = 700$.

1 Introduction

The strictly convex quadratic program

$$\min_{x \in \mathbb{R}^n} \ \tfrac{1}{2} x^\mathsf{T} G x - a^\mathsf{T} x \qquad \text{subject to} \qquad C^\mathsf{T} x \geq b, \tag{1}$$

with $G \in \mathbb{R}^{n \times n}$ symmetric positive definite, $C \in \mathbb{R}^{n \times m}$ and the first m_{eq} constraints understood as equalities, is among the most thoroughly studied problems in numerical optimisation (Nocedal and Wright, 2006; Fletcher, 1987; Boyd and Vandenberghe, 2004). It is also among the most frequently solved in practice: mean–variance portfolio selection (Markowitz, 1952) is exactly (1), and so is every step of a sequential quadratic programming method and every horizon of a linear model predictive controller.

For large sparse instances the field has moved decisively towards operator splitting (Stellato et al., 2020) and interior-point methods. For the dense small-to-medium regime, however — say n from a handful to a few thousand, with m of the same order — active-set methods retain two decisive advantages: they terminate at an exactly feasible vertex of the active-set lattice rather than approaching it asymptotically, and they warm-start almost perfectly, which is why parametric solvers such as qpOASES (Ferreau et al., 2014) are built on them.

Within that regime the Goldfarb–Idnani (GI) dual method (Goldfarb and Idnani, 1982, 1983) is the natural choice. Its distinguishing property is that it maintains *dual* feasibility from the first iterate and drives primal infeasibility to zero, rather than the reverse. Because the unconstrained minimiser $G^{-1}a$ is trivially dual feasible, the method needs no phase-one problem at all — a considerable practical simplification, and the reason GI is preferred over primal active-set methods for this problem class.

1.1 The reference implementation and its lineage

The canonical implementation has an unusually well-documented pedigree. Powell distributed ZQPCVX (Powell, 1983) shortly after the paper appeared; Turlach translated and repackaged the method as the Fortran core of the R package `quadprog` (Turlach et al., 2019), which remains the standard R interface; that Fortran was subsequently transliterated to C and wrapped with Cython (Behnel et al., 2011) as the Python package `quadprog` (McGibbon and contributors, 2024). The numerical core is recognisably the same code in all three, down to variable names and the comment quoting Powell on the machine epsilon.

That code is good code, and it is fast. But it is compiled code, and it carries the consequences: a source install needs a C compiler, wheels must be built for every platform and Python version, and the implementation is opaque to the majority of its users, who are Python programmers rather than C programmers.

1.2 Contribution

We reimplement GI using only NumPy (Harris et al., 2020) and SciPy (Virtanen et al., 2020). The motivation is partly practical — no compiler, no build step, and a solver that can be read and modified by the people who use it — but the reimplementation also creates a controlled experiment. The reference makes a series of implementation choices which are natural in Fortran or C and which have been carried forward for four decades without, as far as we are aware, being revisited. Reimplementing in a language with completely different cost structure forces each of them to be re-derived, and it turns out that some are essential to the algorithm, some are essential for reasons other than the ones usually given, and some are artefacts.

Concretely, this paper contributes:

- a proof that the constraint-insertion update may use any orthogonal reduction — Householder or Givens — without changing the sequence of iterates the solver visits (§4), together with the empirical confirmation that iteration counts match the reference exactly on every problem tested;
- the observation that the reference's packed triangular storage is a performance feature rather than a memory feature, because it is what makes the active submatrix contiguous and hence eligible for a BLAS packed triangular solve (§5);
- a per-column structure detection that reduces three per-iteration products to indexing when a constraint is a bound (§6);
- a differential validation methodology which compares not merely minimisers but complete active-set trajectories against the reference (§7.2);
- two negative results, reported with the measurements that killed them (§8).

2 The dual active-set method

2.1 Optimality conditions

Let $\mathcal{A} \subseteq \{1, \dots, m\}$ index a working set of constraints and write $A = C_{\cdot\mathcal{A}}$ for the matrix of their normals. The Karush–Kuhn–Tucker conditions for (1) require a primal–dual pair (x, u) with

$$Gx - a = Cu, \tag{2}$$

$$C^{\mathsf{T}}x - b \geq 0, \qquad u_i \geq 0 \;\; (i > m_{\mathrm{eq}}), \tag{3}$$

$$u_i\,(C^{\mathsf{T}}x - b)_i = 0 \quad \text{for all } i. \tag{4}$$

A primal active-set method maintains (3) and works towards (2); the dual method maintains (2), (4) and the sign conditions, and works towards primal feasibility.

2.2 One iteration

GI keeps an iterate x that minimises the objective subject to the working-set constraints holding with equality, together with the corresponding multipliers u. If every constraint is satisfied at x, the KKT conditions hold and the method stops. Otherwise it selects the most violated constraint — scaled by the norm of its normal, so that the choice is invariant to how each constraint happens to be written — and moves x towards that constraint's boundary.

Two step lengths compete. The primal step t_2 brings the violated constraint's slack to zero. The dual step t_1 is the largest step for which no multiplier of an active inequality goes negative. If $t_1 < t_2$ the method takes a partial step, drops the constraint whose multiplier reached zero, and repeats; otherwise it takes the full step and adds the new constraint to the working set. Since the objective increases monotonically and each iteration adds a constraint, the method terminates finitely.

The linear algebra per iteration is where all the cost lies. Writing $G^{-1} = JJ^{\mathsf{T}}$ with J upper triangular — so $J = R_G^{-1}$ for the Cholesky factor $G = R_G^{\mathsf{T}}R_G$ — and letting n_* denote the normal of the entering constraint, the method needs

$$d = J^{\mathsf{T}}n_*, \tag{5}$$

$$z = J_2 d_2, \tag{6}$$

$$r = R^{-1}d_1, \tag{7}$$

where $J = [\,J_1\ J_2\,]$ and $d = (d_1, d_2)$ are split at the size k of the working set, and R is the upper triangular factor of the QR factorisation of $J^{\mathsf{T}}A$. Here z is the primal search direction — the component of the constraint normal orthogonal, in the G^{-1} metric, to the active constraints — and $-r$ is the corresponding direction for the multipliers.

The essential structural fact, and the reason GI is efficient, is that

$$J^{\mathsf{T}}A = \begin{bmatrix} R \\ 0 \end{bmatrix} \tag{8}$$

can be *updated* rather than recomputed when a column enters or leaves A. Updating costs $O(n^2)$; recomputing costs $O(nk^2)$. Over a run of $O(n)$ iterations this is the difference between $O(n^3)$ and $O(n^4)$, and we quantify it in §8.

Remark. Equation (7) is a least-squares solve in disguise. Since $A^{\mathsf{T}}G^{-1}A = (J^{\mathsf{T}}A)^{\mathsf{T}}(J^{\mathsf{T}}A) = R^{\mathsf{T}}R$ and $A^{\mathsf{T}}G^{-1}n_* = R^{\mathsf{T}}d_1$,

$$r = R^{-1}d_1 = (R^{\mathsf{T}}R)^{-1}R^{\mathsf{T}}d_1 = (A^{\mathsf{T}}G^{-1}A)^{-1}A^{\mathsf{T}}G^{-1}n_* = \arg\min_{y} \|(J^{\mathsf{T}}A)y - J^{\mathsf{T}}n_*\|_2, \tag{9}$$

and z is its residual mapped back through J. It is tempting to conclude that a library least-squares routine should simply be called here; §8 shows why that costs a factor of 57.

3 Reimplementation in an interpreted setting

3.1 What changes

The reference is written for a machine model in which a scalar operation is cheap and the unit of composition is the loop. NumPy inverts this: a scalar operation reached through the interpreter costs on the order of a microsecond, while an operation on a whole array costs almost the same. Every loop over elements must therefore become an array expression, and the design question becomes not "how many floating-point operations?" but "how many array operations, and does each reach a tuned kernel?"

Two consequences follow immediately. First, the reference's hand-written `dot` and `axpy` — a private BLAS level-1 (Lawson et al., 1979) in `linear-algebra.c` — should be replaced not by NumPy equivalents but by calls that reach the vendor BLAS level-2 (Dongarra et al., 1988) and LAPACK (Anderson et al., 1999). Second, any part of the algorithm that is irreducibly sequential in the number of *iterations* rather than in the number of elements will remain interpreted, and will dominate for small n. Both predictions are borne out (§7.5).

3.2 Design constraints

We adopt three constraints. The public signature is kept identical to (McGibbon and contributors, 2024), including the argument order, the convention that the linear term is subtracted, the column-wise $\geq$ form of the constraints, and the text of the error messages, so that the package is a drop-in replacement. Input arrays are never modified, unlike the reference which destroys G and a. And the dependency set is exactly NumPy and SciPy: no compiler, no Cython, no build step.

4 Householder versus Givens for the insertion

4.1 Two reductions

When a constraint enters the working set, the vector d of (5) must be reduced so that its components beyond the k-th vanish, and the same orthogonal transformation must be applied to the columns of J so that (8) is preserved.

The reference does this with a chain of Givens rotations (Givens, 1958): one rotation per trailing component, each mixing two adjacent columns of J over all n rows. In C this is excellent — the inner loop is tight, the rotations are applied in place, and the arithmetic is minimal. In an interpreted setting it is close to the worst possible structure, because it is $O(n)$ interpreter round-trips per insertion, each doing only $O(n)$ arithmetic. Profiling our first faithful transliteration put 85% of runtime at $n = 150$ in exactly this loop.

The alternative is a single Householder reflection (Householder, 1958),

$$W = I - \frac{2}{\|v\|^2} v v^{\mathsf{T}}, \qquad v = d_{k:} - \alpha e_1, \qquad \alpha = \pm \|d_{k:}\|, \tag{10}$$

which achieves the same reduction in one matrix–vector product and one rank-one update, independently of $n - k$. Both reach the BLAS: `gemv` and `ger` respectively, the latter writing directly into J's buffer so that no $O(n(n-k))$ temporary is allocated. We take the sign of α from d_k, matching the reference's convention, and form the leading entry of v by the cancellation-free rearrangement

$$v_1 = d_k - \operatorname{sign}(d_k)\|d_{k:}\| = \frac{-\operatorname{sign}(d_k)\,\|d_{k+1:}\|^2}{|d_k| + \|d_{k:}\|}, \tag{11}$$

which avoids the subtractive cancellation that the direct formula suffers when $d_{k:}$ is already nearly a positive multiple of e_1 (Golub and Van Loan, 2013).

Deletion keeps the Givens chase, and the reason no analogous single-transformation form exists is worth making precise. Removing a column shifts the remainder left and leaves a single subdiagonal in the trailing block. A Householder reflection annihilates all but one component of a *single* vector; those subdiagonal entries lie in distinct columns, so no one reflection reaches them. The chase is additionally sequential — each rotation's parameters are read from entries the previous rotation wrote — which rules out computing them in a batch.

What the chase does admit is the same routing to the BLAS. Each step applies a 2×2 transformation to a pair of adjacent columns of J; spelled out in NumPy this allocates two n-vectors per step, while BLAS `rot` performs it in place. Because J is stored Fortran-ordered the two columns are contiguous, so the call writes through to J's own buffer. The pairwise update is 2.0 to 3.0 times faster for n between 50 and 2000, and a complete solve is 5 to 8% faster on problems whose working set churns; where no constraint is ever dropped the change is inert. We note that this is a smaller effect than either §5 or §6, and that we found it only after a first measurement showed no gain at all — on a test set in which, as profiling established, no deletion ever occurred.

A detail here corroborates Proposition 1 a second time, on the deletion side. BLAS offers no 2×2 reflection, only the rotation $\left[\begin{smallmatrix} c & -s \\ s & c \end{smallmatrix}\right]$, whose second output is the exact negation of the reflection's; one sign flip therefore recovers the reference's convention. Proposition 1 says that flip is unnecessary, since using the rotation outright replaces R by $S_k R$ and J by JS. We implemented that variant and confirmed it: every iteration count in §7.2 is reproduced, with $|R|$ and $|J|$ elementwise unchanged and only signs differing. It is not retained. It measures no faster — the flip is one in-place pass — and it forfeits the symmetry of the reflection, which is what allows one 2×2 to serve both J's columns and R's rows; the rotation requires its transpose on the R side and turns the degenerate case into a signed exchange.

4.2 The transformations differ; the iterates do not

Householder and Givens reductions do not produce the same Q and R: signs along the diagonal of R, and hence the signs of some columns of J, differ. Since the solver consumes R and J, one might expect the substitution to perturb the computation. It does not, and the reason is worth stating precisely.

Proposition 1 (Invariance of the iterates)**.** *Let $\hat{J} = JS$ and $\hat{R} = S_k R$ where $S = \operatorname{diag}(s)$ with $s_i \in \{-1, +1\}$ and S_k is its leading $k \times k$ block. Then the quantities r and z of (7) and (6) are unchanged, and in IEEE arithmetic they are unchanged* exactly*.*

Proof. From (9), $r = (A^\mathsf{T} G^{-1} A)^{-1} A^\mathsf{T} G^{-1} n_*$ depends only on A, n_* and G, and none of these is altered by the choice of reduction. Directly: $\hat{d} = \hat{J}^\mathsf{T} n_* = Sd$, so $\hat{r} = \hat{R}^{-1}\hat{d}_1 = (S_k R)^{-1} S_k d_1 = R^{-1} d_1 = r$. For the primal direction, $\hat{z} = \hat{J}_2 \hat{d}_2 = \sum_{j>k} (s_j J_{\cdot j})(s_j d_j) = \sum_{j>k} s_j^2 J_{\cdot j} d_j = z$. Both cancellations are multiplications by ± 1, which are exact in IEEE 754, so the identities hold in floating point and not merely in exact arithmetic. □

The proposition explains why the substitution is safe, but it does not by itself guarantee that the two implementations follow the same trajectory: rounding in the reductions themselves still differs, and the algorithm's branches — which constraint is most violated, whether $t_1 \geq t_2$ — are discontinuous functions of those roundings. That the trajectories coincide anyway is an empirical claim, and we test it directly in §7.2 by comparing iteration counts rather than only solutions.

Remark. Proposition 1 also disposes of a natural objection to the whole enterprise. One might ask why the solver maintains J, an $n \times n$ dense matrix, rather than the orthogonal factor Q of $J^\mathsf{T} A$ together with the triangular R_G. The answer is that J folds R_G^{-1} and the accumulated Q into a single matrix, so that (5) and (6) are each one `gemv` regardless of k. Section 8 shows what happens when this folding is undone.

5 Storage as a performance decision

The reference stores R as packed columns: entry (i, j), $i \le j$, at flat offset $j(j+1)/2 + i$. The obvious reading is that this halves memory, and our first reimplementation accordingly replaced it with a dense (r, r) array, documenting the change as affecting "only the addressing". That was wrong, and the way in which it was wrong is instructive.

Equation (7) is solved once per iteration against the leading $k \times k$ block of R. In a dense array that block is a *strided* view: its column stride is r, not k. Handed such a view, SciPy's LAPACK wrapper cannot pass it through and copies it, at a cost of $O(k^2)$ memory traffic per iteration. In packed storage the same block is the leading $k(k+1)/2$ entries — contiguous by construction, at every k — and BLAS `tpsv` reads it in place.

The effect is out of all proportion to the arithmetic. At $n = 700$ with $k = 383$:

Triangular solve variant	Time (μs)
`trtrs` on the strided view (dense R)	77.0
`trtrs` on a contiguous copy	22.6
`tpsv` on the packed triangle	7.5

A factor of 10.2, none of it arithmetic. The giveaway in the profile was that this solve was running at roughly 1.2 Gflop/s while a neighbouring `gemv` on the same data managed 10 Gflop/s. Over a complete solve the operation fell from 15.5 ms (21% of runtime) to 2.0 ms (3.5%).

The cost is borne by the deletion routine, which mixes two *rows* of R across a range of columns. Because column offsets grow with the column index, what is a contiguous slice in the dense layout becomes a gather in the packed one. Insertion is unaffected: a column is one contiguous run in either layout.

We draw a general moral. In compiled numerical code, data layout is chosen for cache behaviour and for memory. In array-language numerical code it is additionally, and often dominantly, a question of *which kernels the layout makes reachable*. A layout that forces a copy at the library boundary can cost an order of magnitude on an operation whose flop count is negligible.

6 Exploiting constraint structure

A bound constraint $x_i \ge \ell_i$ is a single nonzero in its column of C, and a box-constrained problem consists of nothing else. When the entering constraint's column is νe_ρ for a scalar ν and index ρ, three of the per-iteration products degenerate:

Quantity	General	Column is νe_ρ
slack $C^\mathsf{T} x$	$O(nm)$ `gemv`	$O(m)$ gather
$d = J^\mathsf{T} n_*$	$O(n^2)$ `gemv`	$O(n)$: one scaled row of J
$z^\mathsf{T} n_*$	$O(n)$ dot	$O(1)$

The slack product matters most, because a box-constrained problem has $m = 2n$ and the product is evaluated once per outer iteration; before this change it was the single largest consumer of arithmetic in the solver.

Detection is performed once, per column. The per-column choice is not a refinement but the point: the case worth optimising is *mixed*. A mean–variance problem (Markowitz, 1952) carries a dense budget column $\sum_i x_i = 1$ alongside $2n$ bounds, and an all-or-nothing test would observe that single dense column and send the entire problem down the general path. The slack evaluation is additionally given a three-way choice — an all-unit gather, a compiled sparse CSR

product, or the dense product — since a mixed problem of this shape has density $O(1/n)$ and is far too sparse to want the last.

Because this is a fast path around arithmetic the general path performs anyway, it cannot alter the answer; we verify that claim rather than assert it (§7.2). One trap deserves naming: an all-zero column must not be mistaken for a unit column. It expresses $0 \geq b_i$, which no x can influence, and treating it as a unit column would scale a row of J by zero and lose the infeasibility verdict.

7 Experiments

7.1 Methodology

All measurements are on an `arm64` machine, CPython 3.12, NumPy 2.5.1, SciPy 1.18, against `quadprog` 0.1.13 built from its published source. Timings are the best of five batches after a warm-up call; we report the best rather than the mean because the quantity of interest is the achievable time and the noise is one-sided.

Random problems are generated with $G = BB^{\mathsf{T}} + nI$ for B Gaussian, which is positive definite by construction and moderately conditioned. Where box constraints are used, the bounds straddle the unconstrained minimiser so that roughly half of them bind at the solution.

7.2 Agreement with the reference

Correctness for a solver of this kind is usually argued by comparing minimisers. That is too weak a test to detect the class of error we are most exposed to: a change that perturbs the active-set trajectory while still converging to the right answer would pass, and would be a latent hazard on degenerate problems. We therefore compare complete trajectories, by requiring that both implementations report identical iteration counts — the number of constraints added and the number dropped — as well as identical minimisers, objectives, multipliers and active sets.

Over 4000 random problems with $2 \leq n \leq 11$, up to 14 constraints and mixed equality counts:

Quantity	Agreement with (McGibbon and contributors, 2024)
Iteration counts (added, dropped)	exact, 3027 / 3027 feasible problems
Infeasibility verdict	exact, 973 / 973 infeasible problems
Minimiser x	max abs. difference 3.0e-09
Objective	max rel. difference 2.5e-12

That the iteration counts agree exactly on every feasible problem, and the infeasibility verdict on every infeasible one, is a considerably stronger statement than agreement on the answer: the two implementations add and drop the same constraints in the same order, despite one using Householder reflections and the other Givens rotations. Proposition 1 explains why this is possible; the experiment establishes that it in fact occurs.

The suite also includes the upstream test problems verbatim, with their published iteration counts asserted, and a differential test at $n = 60, 140, 220$ where the vectorised paths dominate. The implementation carries 1066 tests at 100% line and branch coverage.

7.3 Where the two legitimately differ

Duplicated or linearly dependent constraints make the *dual* solution non-unique: a multiplier may sit on either copy. Both implementations return a valid KKT point but not necessarily the same one, so multipliers and reported active sets differ while x and the objective do not. We detected this as an apparent test failure and, rather than loosen a tolerance, verified that both answers satisfy (2)–(4) to machine precision — the stationarity residual is $\sim 1e-16$ for both —

and changed the test to check the KKT conditions on such problems instead of demanding an identical dual.

7.4 Accuracy

Both implementations accumulate the objective incrementally rather than re-evaluating the quadratic, following (Goldfarb and Idnani, 1983). Measuring each against a direct re-evaluation at *its own* minimiser over 2164 problems, the worst-case drift is 1.5e-8 absolute (7.4e-15 relative) here against 3.7e-8 (1.8e-14) for the reference; but neither dominates problem by problem, the reference being closer on 801 problems and this implementation on 782, with 581 ties.

For the KKT residual proper, over 3000 problems the worst relative stationarity residual $\|Gx - a - Cu\|_\infty$, scaled, is 7.3e-13 here against 8.8e-13 for the reference, with this implementation strictly more accurate on 1035 problems and the reference on 869. The two are therefore of equal numerical quality, as the backward stability of both reductions (Wilkinson, 1965; Higham, 2002) would predict.

Remark. An earlier version of the objective experiment evaluated the exact objective at *this* implementation's minimiser and compared both accumulated values against that single point, which charged the reference for the distance between the two minimisers in addition to its own drift and produced a flattering 1.4e-8 versus 3.1e-7. We record the error because its direction was self-serving and it survived a code change unnoticed, being computed outside the test suite.

7.5 Performance

Box-constrained problems, n variables and $2n$ constraints, per solve:

n	This work (ms)	`quadprog` C (ms)	Ratio
10	0.077	0.006	12.5× slower
25	0.16	0.017	9.4× slower
50	0.40	0.076	5.3× slower
100	0.96	0.60	1.6× slower
200	2.8	5.5	**2.0× faster**
400	11.5	47	**4.1× faster**
800	53	461	**8.8× faster**
1600	374	4121	**11× faster**

The crossover is at $n \approx 135$, located by sweeping the interval: the ratio passes unity between $n = 130$ (1.02×) and $n = 140$ (0.92×).

Below the crossover the cost is interpreter dispatch: roughly 14 μs per iteration spread over some fourteen array operations, against 6 μs for the reference to complete an entire $n = 10$ solve. This is a floor set by the language implementation, not by the algorithm, and §8 confirms it by removing it.

Above the crossover this implementation is the faster of the two, and the reason is not subtle: the reference's inner products and `axpy`s are hand-written scalar loops in `linear-algebra.c`, while the corresponding work here is expressed as BLAS calls that reach vendor-tuned, vectorised kernels. The asymptotic behaviour is $O(n^3)$ for both; the constant differs by an order of magnitude in the vendor's favour.

After both optimisations, the residual profile at $n = 700$ is dominated by the insertion update, at roughly 50% of runtime.

7.6 Attacking the iteration count instead

Every measurement above accepts the dual method's defining feature: it reaches the active set one constraint at a time, so its iteration count grows with the size of that set — 74 outer

iterations at $n = 100$ on a budget-plus-bounds problem. Below the crossover a solve costs very nearly its count of interpreter-level operations, so the floor identified above is a floor on *iterations times dispatch*, and the preceding sections attacked only the second factor.

The first factor yields to a different method. A primal-dual active set guesses the whole active set $\mathcal{A}$ at once, solves the equality-constrained problem it induces,

$$(C_{\mathcal{A}}^{\mathsf{T}} G^{-1} C_{\mathcal{A}})\lambda = b_{\mathcal{A}} - C_{\mathcal{A}}^{\mathsf{T}} G^{-1} a, \qquad x = G^{-1}a + G^{-1}C_{\mathcal{A}}\lambda,$$

and repairs the guess from the signs that come back: an active inequality wanting a negative multiplier leaves the set, a violated inactive constraint joins it. Across every family we measured this settles in two to four repairs *independently of* n. It trades flops, which are abundant at these sizes, for dispatches, which are not.

The method is not globally convergent, so the trade is only sound because the answer is checked. The program is strictly convex, so the KKT conditions are sufficient and not merely necessary: a point satisfying them is *the* minimiser. Every candidate is therefore certified before it is returned, and one that fails is discarded in favour of the exact walk. This is not a formality. Over 1164 converged attempts, two had settled on a set that was not optimal — one of them 0.85 from the true minimiser — and the certificate rejected both; among the points it accepted, the largest disagreement with the exact walk was 3.3×10^{-15}.

n	Certified path (ms)	`quadprog` C (ms)	Ratio
10	0.081	0.006	13.2× slower
25	0.11	0.017	6.4× slower
50	0.14	0.076	1.9× slower
100	0.24	0.60	**2.6× faster**
200	0.56	5.5	**9.7× faster**
400	2.4	47	**19× faster**
800	13.4	461	**34× faster**
1600	61	4121	**68× faster**

The crossover moves from $n \approx 135$ to $n \approx 65$, the ratio passing unity between $n = 60$ (1.21×) and $n = 70$ (0.84×). It lands that early for a reason worth stating: the reference is itself a dual active-set walk, so it too adds one constraint per iteration — roughly $0.45n$ of them on these problems — where the guess converges in about three repairs whatever n is. Each repair is far heavier, but a heavy constant beats a light $O(n)$.

Below twelve variables the path is not attempted: the guess is likeliest to be linearly dependent when there are few variables to spread the active set over, and that is also where it wins least. Measured over 300 instances per size, the certified fraction is 100% from twelve variables upward on all four families, against 23% at $n = 3$ on equality-constrained problems. It is offered as an option rather than a default, because the working-set edits it counts are not the reference's iteration counts, and §7.2 is a claim about those.

8 Two negative results

Both of the following were pursued to working prototypes, verified against the shipped implementation, and measured. We report them because the reasoning that motivated each is more persuasive than the outcome, and because negative results of this kind are ordinarily discarded.

8.1 Leaving the orthogonal factor implicit

Insertion updates J explicitly, at $O(n(n-k))$, and this is now half the runtime. LAPACK's `geqrf`/`ormqr` pair suggests an alternative: store the Householder vectors and never form Q

(Bischof and Van Loan, 1987; Schreiber and Van Loan, 1989). Insertion then costs essentially nothing, because d of (5) *already is* the reduced column — the reflection is read off it and appended, and no matrix is touched. The factorisation is maintained as $J_0 = R_G^{-1}$ together with a chain $H_1 \cdots H_k$, and (5) and (6) become a triangular product followed by an `ormqr` application.

The prototype reproduces the shipped implementation exactly — identical iteration counts on 300 problems, worst $|\Delta x| = 4.7e-12$ — and is between 2.4 and 2.6 times *slower*, with no deletions at all:

n	Explicit J (ms)	Implicit Q (ms)	Ratio
200	3.41	8.56	2.51× slower
400	13.94	33.60	2.41× slower
700	45.30	119.35	2.63× slower

The flop count explains it. Per iteration at working-set size k:

	Explicit J	Implicit Q
d	one `gemv`, $2n^2$	`trmv` n^2 + `ormqr` $\sim 4nk$
z	`gemv`, $2n(n-k)$	`ormqr` $\sim 4nk$ + `trmv` n^2
insert	$4n(n-k)$	free
summed over k	$\sim 5n^3$	$\sim 6n^3$

Removing the insertion does not remove its work; it relocates it. Applying an implicitly stored Q costs $O(nk)$, and the solver applies it *twice per iteration* — precisely the work the explicit update performs *once*. Forming J amortises the accumulated Q into a single dense matrix so that every subsequent application is one `gemv` irrespective of k; that is the entire reason to form it. Implicit storage is advantageous when Q is applied rarely relative to the number of reflections, as in a one-shot factorisation with few right-hand sides. Here it is applied twice per reflection, which is its worst case.

Deletion supplies a second, independent objection. A Givens chase cannot be absorbed into a stored Householder chain, so a deletion becomes a refactorisation: 82% of runtime on a problem with 200 of them, and 2.48 times slower overall. We measured deletion frequency across problem classes — 0% of steps for box and budget-plus-bounds problems, 2.2% for random dense C, 45% for an adversarial case constructed to force churn — and concluded that a hybrid would have been viable had the insertion side won. It does not.

Remark (Library routines). SciPy ships compiled routines for precisely this updating problem, namely `qr_insert`, `qr_delete` and `qr_update`. Over the full sequence of 383 column insertions at $n = 700$ they take 39.5 ms, against 31.3 ms here. The difference is that the library API returns fresh arrays, and so reallocates a 3.9 MB Q on every call. Solving (9) from scratch each iteration with `lstsq` costs 11.5 ms at $k = 383$ against 0.5 ms for the update — which, over the run, is the $O(n^4)$ versus $O(n^3)$ distinction of §2, and amounts to a factor of roughly 57.

8.2 Compiling with Numba

The small-n deficit of §7.5 is dispatch overhead, and a just-in-time compiler removes dispatch. We ported the solver to Numba (Lam et al., 2015), giving it the best available shot: both matrix–vector products are expressed so that their arguments remain contiguous, which is what allows Numba to route them to BLAS `gemv` exactly as the NumPy version does, and compilation is cached so that JIT time is excluded.

The port is faithful — identical iteration counts on 300 problems, worst $|\Delta x| = 8.6e-12$.

	D		ense C (ms)	B		ox (ms)
n	NumPy	Numba		NumPy	Numba	
10	0.022	0.004	5.9× faster	0.08	0.01	13.9× faster
50	0.051	0.051	—	0.55	0.14	3.8× faster
100	0.094	0.133	1.4× slower	1.53	0.68	2.2× faster
200	0.278	0.484	1.7× slower	3.48	3.15	1.1× faster
400	1.180	2.039	1.7× slower	13.13	23.79	1.8× slower
700	4.435	8.893	2.0× slower	43.95	110.5	2.5× slower

At $n = 10$ the compiled version is between 1.04 and 1.33 times faster than the C extension, where the NumPy version is 12.5 times slower. The interpreter floor is therefore not a property of the algorithm. Above the crossover — about $n = 50$ dense, $n = 250$ box — Numba loses, because the rank-one update becomes LLVM-generated loops where the NumPy version calls a vendor `ger`, and that update is half of large-n runtime.

We document rather than adopt this. Numba's `llvmlite` is a 38 MB dependency which pins NumPy backwards; it contradicts the package's central claim of needing no compiler; it requires a second copy of a numerically delicate loop, kept differentially tested against the first; and the sizes at which it wins are those where every implementation is already fast in absolute terms, 0.08 ms against 0.006 ms.

9 Discussion

Three observations generalise beyond this solver.

Layout determines which kernels are reachable. The packed-storage result (§5) was worth a factor of ten on its operation and none of it was arithmetic. In compiled code, data layout is chosen for cache behaviour; in array-language code it additionally decides whether a library call can proceed in place or must copy at the boundary. A strided view handed to a wrapper that requires contiguity is a silent $O(k^2)$ tax per call. We had inherited the right layout from the reference, discarded it as a mere memory optimisation, and had to rediscover its purpose by profiling.

Flop counts mislead in both directions. Our initial accounting identified the slack product as 44% of arithmetic at $n = 400$ and predicted a large gain from eliminating it; the realised gain was 1.35 times, because the eliminated work was vectorised and therefore cheap per flop. Conversely the triangular solve was a negligible 0.5% of arithmetic and 21% of runtime. In the implicit-Q experiment the flop count was the correct guide only once it was computed properly, at which point it predicted the observed loss. The reliable procedure is to measure per-operation wall-clock inside the real iteration, which is what eventually located both wins.

Invariance arguments license aggressive substitution. Proposition 1 is elementary, but without it the Householder substitution would have been an unjustifiable risk in a solver whose branches are discontinuous in its own rounding. Identifying which quantities an iterative method actually consumes — as opposed to which it happens to compute — is what makes it safe to change the representation underneath. We would expect similar arguments to license similar substitutions in other active-set and projection methods.

9.1 Limitations

The implementation targets dense problems and inherits GI's assumption that G is positive definite; semidefinite G requires regularisation or a different method. Small problems remain

slower than the compiled reference by up to 13 times, and §8 establishes that closing that gap requires leaving pure NumPy. The structure detection of §6 recognises single-nonzero columns and general sparsity in the slack product but does not exploit sparsity in G, for which a different factorisation strategy would be appropriate. Finally, our differential validation is against one reference implementation; agreement on trajectories is strong evidence of faithfulness to that implementation, not proof of correctness in the abstract.

10 Conclusion

The Goldfarb–Idnani method is forty years old, and its canonical implementation is scarcely younger. Reimplementing it in an array language proved to be a useful way of separating the algorithm from its Fortran-era encoding. The chain of Givens rotations turned out to be replaceable, and provably so. The packed triangular storage turned out to be indispensable, though not for the reason it is usually given. The one structural fact the reference does not exploit — that a bound constraint is a single nonzero — was worth another 25%. Two further attractive ideas turned out to be worth nothing at all, and we have reported them at the same length as the successes.

The resulting solver needs no compiler and is, for $n \gtrsim 135$, several times faster than the compiled code it reimplements.

Reproducibility

The implementation, the complete test suite, and the experiment scripts underlying every table in this paper are available at `https://github.com/Jebel-Quant/quadprog`. The differential tests against (McGibbon and contributors, 2024) run in continuous integration on CPython 3.11, 3.12, 3.13 and 3.14, on Linux, macOS and Windows.

References

E. Anderson, Z. Bai, C. Bischof, S. Blackford, J. Demmel, J. Dongarra, J. Du Croz, A. Greenbaum, S. Hammarling, A. McKenney, and D. Sorensen. *LAPACK Users' Guide*. SIAM, 3rd edition, 1999. doi: 10.1137/1.9780898719604.

Stefan Behnel, Robert Bradshaw, Craig Citro, Lisandro Dalcin, Dag Sverre Seljebotn, and Kurt Smith. Cython: The best of both worlds. *Computing in Science & Engineering*, 13(2):31–39, 2011. doi: 10.1109/MCSE.2010.118.

Christian Bischof and Charles Van Loan. The WY representation for products of Householder matrices. *SIAM Journal on Scientific and Statistical Computing*, 8(1):s2–s13, 1987. doi: 10.1137/0908009.

Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.

Jack J. Dongarra, Jeremy Du Croz, Sven Hammarling, and Richard J. Hanson. An extended set of FORTRAN basic linear algebra subprograms. *ACM Transactions on Mathematical Software*, 14(1):1–17, 1988. doi: 10.1145/42288.42291.

Hans Joachim Ferreau, Christian Kirches, Andreas Potschka, Hans Georg Bock, and Moritz Diehl. qpOASES: a parametric active-set algorithm for quadratic programming. *Mathematical Programming Computation*, 6(4):327–363, 2014. doi: 10.1007/s12532-014-0071-1.

Roger Fletcher. *Practical Methods of Optimization*. Wiley, 2nd edition, 1987.

Wallace Givens. Computation of plane unitary rotations transforming a general matrix to triangular form. *Journal of the Society for Industrial and Applied Mathematics*, 6(1):26–50, 1958. doi: 10.1137/0106004.

Donald Goldfarb and Ashok Idnani. Dual and primal-dual methods for solving strictly convex quadratic programs. In *Numerical Analysis*, volume 909 of *Lecture Notes in Mathematics*, pages 226–239. Springer, 1982. doi: 10.1007/BFb0092976.

Donald Goldfarb and Ashok Idnani. A numerically stable dual method for solving strictly convex quadratic programs. *Mathematical Programming*, 27(1):1–33, 1983. doi: 10.1007/BF02591962.

Gene H. Golub and Charles F. Van Loan. *Matrix Computations*. Johns Hopkins University Press, 4th edition, 2013.

Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, et al. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020. doi: 10.1038/s41586-020-2649-2.

Nicholas J. Higham. *Accuracy and Stability of Numerical Algorithms*. SIAM, 2nd edition, 2002. doi: 10.1137/1.9780898718027.

Alston S. Householder. Unitary triangularization of a nonsymmetric matrix. *Journal of the ACM*, 5(4):339–342, 1958. doi: 10.1145/320941.320947.

Siu Kwan Lam, Antoine Pitrou, and Stanley Seibert. Numba: a LLVM-based Python JIT compiler. In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC (LLVM '15)*, pages 7:1–7:6, 2015. doi: 10.1145/2833157.2833162.

C. L. Lawson, R. J. Hanson, D. R. Kincaid, and F. T. Krogh. Basic linear algebra subprograms for Fortran usage. *ACM Transactions on Mathematical Software*, 5(3):308–323, 1979. doi: 10.1145/355841.355847.

Harry Markowitz. Portfolio selection. *The Journal of Finance*, 7(1):77–91, 1952. doi: 10.2307/2975974.

Robert T. McGibbon and contributors. quadprog: Quadratic programming solver for Python. https://github.com/quadprog/quadprog, 2024.

Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, 2nd edition, 2006. doi: 10.1007/978-0-387-40065-5.

M. J. D. Powell. ZQPCVX: a FORTRAN subroutine for convex quadratic programming. Technical Report DAMTP/1983/NA17, Department of Applied Mathematics and Theoretical Physics, University of Cambridge, 1983.

Robert Schreiber and Charles Van Loan. A storage-efficient WY representation for products of Householder transformations. *SIAM Journal on Scientific and Statistical Computing*, 10(1):53–57, 1989. doi: 10.1137/0910005.

Bartolomeo Stellato, Goran Banjac, Paul Goulart, Alberto Bemporad, and Stephen Boyd. OSQP: an operator splitting solver for quadratic programs. *Mathematical Programming Computation*, 12(4):637–672, 2020. doi: 10.1007/s12532-020-00179-2.

Berwin A. Turlach, Andreas Weingessel, and Cleve Moler. *quadprog: Functions to Solve Quadratic Programming Problems*, 2019. URL https://CRAN.R-project.org/package=quadprog. R package; Fortran implementation of the Goldfarb–Idnani dual method.

Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3): 261–272, 2020. doi: 10.1038/s41592-019-0686-2.

James H. Wilkinson. *The Algebraic Eigenvalue Problem*. Clarendon Press, Oxford, 1965.